

Product Requirements Document (PRD)

Heart Disease Risk Prediction App

Product	Heart Disease Risk Prediction (educational prototype)
Version	2.0 (Advanced interactive version)
Author	Shaurya Singh — solo project
Institution	ABES Engineering College, Ghaziabad (III-Year B.Tech, CSE-AIML)
Timeline	29 Sep 2026 – 31 Oct 2026 (4 sprints)
Status	Completed

1. Overview

An interactive web app where a person enters basic medical details (age, blood pressure, cholesterol, chest pain type, etc.) and **instantly sees the estimated likelihood of heart disease** as a live gauge with a risk level. The app also explains **why** the model gave that result, shows **how the person compares** with other patients, lets them try **what-if changes**, and produces a **PDF report**.

Beyond the individual result, the app includes a **Model Insights** dashboard, a **Data Explorer**, **batch prediction** (editable table or CSV upload), a **session history** and a searchable **glossary**.

It is built with Python, Pandas, scikit-learn, Plotly, fpdf2 and Streamlit. It has **no database, no login and no separate backend**. It is an **educational prototype, not a diagnostic tool**.

2. Problem Statement

Heart disease is a leading cause of death, yet many people do not understand how their own health numbers (blood pressure, cholesterol, exercise response) relate to heart risk. Clinical tools are complex, and most students learn machine learning without ever seeing a model used by a real person.

This project shows how a trained ML model can be turned into a friendly, understandable web tool that raises awareness and encourages people to seek medical advice.

3. Goals and Non-Goals

Goals

1. Give an instant, live prediction as the user changes any input (result within 2 seconds).
2. Explain every result in plain language: risk level, factors that raise or lower it, comparison with other patients.
3. Let users explore cause and effect safely through a what-if simulator and risk curve.
4. Achieve at least 80% test accuracy and a cross-validated ROC-AUC of at least 0.85 (current: about 83–88% accuracy, CV ROC-AUC about 0.90).
5. Provide transparency for evaluators: tuned models, honest model selection, ROC curves, confusion matrix, feature importance.
6. Make the app friendly for non-technical users: grouped inputs, tooltips, example profiles, simple view and glossary.
7. Keep setup simple: install libraries and run one command.

Non-Goals

- Medical diagnosis or treatment advice
- User accounts, login or a database
- Separate API backend or React frontend

- Multilingual interface
- Real-time data from wearables or hospitals

4. Target Users

Persona	Description	Need
Curious user	Adult with no medical background	Understand if their numbers suggest heart risk and whether to see a doctor
Student / evaluator	Peer, faculty or guide reviewing the project	See a working, well-explained ML application

5. User Stories

ID	Story	Priority
U1	As a user, I want a friendly form with sliders and tooltips so that entering my details is easy.	High
U2	As a user, I want the result to update instantly as I change values so that I can see cause and effect.	High
U3	As a user, I want one-click example profiles so that I can try the app quickly.	Medium
U4	As a user, I want to know why I got this result so that I can trust and understand it.	High
U5	As a user, I want to compare myself with other patients so that I know where I stand.	Medium
U6	As a user, I want a what-if simulator so that I can see how changing blood pressure or cholesterol might affect the estimate.	Medium
U7	As a user, I want tips, disease information and a PDF report so that I can discuss the result with a doctor.	Medium
U8	As a user, I want a clear disclaimer so that I don't mistake the result for a diagnosis.	High
U9	As an evaluator, I want to see model comparison, ROC curves, confusion matrix and feature importance so that I can judge credibility.	Medium
U10	As a user, I want to explore the dataset and run predictions for many people at once so that I can learn more from the data.	Medium
U11	As a user, I want a simple view and a glossary so that the app is not overwhelming.	Low

6. Features and Requirements

6.1 Functional Requirements

ID	Requirement	Priority
FR-1	Grouped input form (About you, Vitals and blood tests, Symptoms, Heart tests) for 13 medical details, using sliders and dropdowns with tooltips and live hints (blood pressure and cholesterol category, expected maximum heart rate)	High
FR-2	Live result: likelihood %, risk level (Low < 30%, Moderate 30–60%, High ≥ 60%), gauge with threshold marker, and number of models that agree	High
FR-3	Example profiles (low, moderate, high) and a Reset button	Medium
FR-4	List of values that stand out (e.g. blood pressure ≥ 140, cholesterol ≥ 240); disease information and advice to see a doctor when likelihood is at or above the threshold	Medium
FR-5	Disclaimer displayed at the top at all times	High
FR-6	"Why this result?" chart showing how each input raises or lowers the likelihood, plus a table of all model predictions	High

ID	Requirement	Priority
FR-7	"How you compare" radar chart against typical patients with and without disease, and percentiles of key values	Medium
FR-8	What-if simulator (blood pressure, cholesterol, heart rate, blood sugar) and risk curve for a chosen factor	Medium
FR-9	Personalised lifestyle tips and a downloadable one-page PDF report	Medium
FR-10	Sidebar settings: prediction model, decision threshold, simple or detailed view	Medium
FR-11	Model Insights tab: comparison table and metric chart, ROC curves, confusion matrix with threshold explorer, feature importance	Medium
FR-12	Data Explorer tab: age and sex filters, distribution and scatter plots (with a "You" marker), correlation heatmap, data download	Medium
FR-13	Batch prediction through an editable table and CSV upload, with downloadable results	Medium
FR-14	Session history: save results, view table and trend chart, download CSV (kept only in the browser session)	Medium
FR-15	About page with limitations and a searchable glossary	Low

6.2 Machine Learning Requirements

ID	Requirement
ML-1	Load UCI Heart Disease (Cleveland) data with Pandas; remove duplicates and invalid values (ca = 4, thal = 0); set label 1 = disease
ML-2	Scale numeric features and one-hot encode categorical features inside a scikit-learn Pipeline
ML-3	Tune Logistic Regression, Random Forest, SVM and Gradient Boosting with GridSearchCV (5-fold cross-validation, scored by ROC-AUC)
ML-4	Build a soft-voting ensemble of the tuned models
ML-5	Stratified 80/20 split; evaluate with accuracy, precision, recall, F1, ROC-AUC and confusion matrix
ML-6	Select the default model by cross-validated ROC-AUC, not by test results
ML-7	Compute permutation feature importance and a typical no-disease reference profile for explanations
ML-8	Save all models and artifacts with Joblib

6.3 Non-Functional Requirements

Area	Requirement
Performance	Result updates within 2 seconds of any input change
Usability	Tabbed single-page interface, tooltips, example profiles, simple view and glossary; works on desktop and mobile browsers
Privacy	No database; inputs and history exist only in the browser session
Portability	Runs on Windows, Linux, macOS with Python 3.9+ and Streamlit 1.50+, or on Streamlit Community Cloud
Reliability	Inputs restricted to valid ranges; models retrain automatically if the saved file is missing
Transparency	Explanations, model comparison and limitations are visible in the app

7. User Flow

1. User opens the app and reads the disclaimer (optionally opens "How to use this app").
2. User loads an example profile or sets values in the four input cards.
3. The result updates live: risk badge, gauge, likelihood, models agreeing, values that stand out.

4. User explores **Why this result**, **How you compare** and **What-if & risk curve**, or turns on the simple view.
5. User saves the result to history and/or downloads the **PDF report**.
6. User may open Model Insights, Data Explorer, Batch & History or the Glossary.

8. Data and Model

Dataset	UCI Heart Disease (Cleveland), 14-column version
Raw size	303 records
After cleaning	296 records (136 disease, 160 no disease)
Features	13 (6 numeric, 7 categorical)
Models	Logistic Regression, Random Forest, SVM, Gradient Boosting (each tuned with GridSearchCV) and a soft-voting ensemble
Selection rule	Highest cross-validated ROC-AUC on the training set
Selected model	SVM (CV ROC-AUC about 0.906)

Model	CV ROC-AUC	Accuracy	Precision	Recall	F1	Test ROC-AUC
Logistic Regression	0.900	0.833	0.875	0.750	0.808	0.961
Random Forest	0.898	0.867	0.885	0.821	0.852	0.958
SVM (selected)	0.906	0.883	0.920	0.821	0.868	0.960
Gradient Boosting	0.885	0.867	0.857	0.857	0.857	0.943
Voting Ensemble	0.903	0.867	0.885	0.821	0.852	0.960

Explainability method: each input is replaced by the typical value of patients without heart disease and the change in likelihood is measured. This works with any model and is an approximation, not a causal explanation.

Known limitations: small dataset (about 300 patients), one hospital source, older data, and category codes in the CSV that are inferred rather than documented. The test set has only about 60 patients, so test scores are optimistic and vary between splits; cross-validated scores are a more realistic estimate.

9. Technical Approach

```
heart.csv → Pandas cleaning → scikit-learn preprocessing pipeline
→ Tune 4 models (GridSearchCV) + voting ensemble
→ Pick best by cross-validated ROC-AUC → save artifacts (Joblib)
→ Streamlit app: live prediction, explanation, what-if,
    PDF report, insights, explorer, batch and history
```

Technology	Needed?	Purpose
Python	Yes	Main language
Pandas, NumPy	Yes	Load and clean data
scikit-learn	Yes	Preprocessing, tuning, training, evaluation, importance
Joblib	Yes	Save/load models and artifacts
Streamlit	Yes	Web interface
Plotly	Yes	Interactive charts
fpdf2	Yes	PDF report

Technology	Needed?	Purpose
React, Node/Express, Flask/FastAPI	Optional	Future custom frontend/API
MongoDB / SQLite	Optional	Future accounts and history

Files: app.py (interface), train.py (training), utils.py (helpers, explanations, PDF), heart.csv, requirements.txt, README.md.

Hosting: local (streamlit run app.py) or Streamlit Community Cloud via GitHub.

10. Success Metrics

Metric	Target
Cross-validated ROC-AUC	≥ 0.85 (currently about 0.90)
Test accuracy	$\geq 80\%$ (currently about 83–88%)
Test recall (disease cases caught)	$\geq 75\%$ (currently about 75–86%)
Live update time	≤ 2 seconds
App crashes on valid input	0
Feature checks	Example profiles, live updates, what-if, PDF, batch and history all work without errors
Deliverables	Working demo, README, SRS, PRD, synopsis and sprint plan completed by 31 Oct 2026

11. Milestones

Sprint	Dates	Goal
1	29 Sep – 05 Oct 2026	Environment ready; dataset loaded, explored and cleaned
2	06 – 12 Oct 2026	Preprocessing pipeline; four models tuned, ensemble trained, best model evaluated and saved
3	13 – 19 Oct 2026	Live interactive risk assessment with explanation and comparison
4	20 – 31 Oct 2026	What-if, PDF report, insights, explorer, batch and history; testing, documents and demo

Plan size: 22 user stories, 67 story points and about 97 hours of work for one developer (about 20 hours per week).

12. Risks and Mitigations

Risk	Impact	Mitigation
Small dataset causes unstable or overfitted results	High	GridSearchCV with 5-fold CV, model selection by CV score, limitations stated
Users mistake the result for a diagnosis	High	Prominent disclaimer, careful wording, advice to see a doctor
Wrong category labels (dataset codes are inferred)	Medium	Verify against dataset documentation before the demo
Windows Smart App Control blocks Python libraries	Medium	Turn off the setting, use WSL, Google Colab or Streamlit Cloud
Scikit-learn version mismatch with saved .pkl	Low	App retrains automatically when the model file is missing

Risk	Impact	Mitigation
Solo workload with a larger feature set	Medium	Prioritise core stories first; Sprint 4 features are Medium/Low priority and can be dropped
Feature creep makes the interface overwhelming	Medium	Simple/detailed view toggle, example profiles and glossary
Explanations misread as medical causes	High	Plain wording, limitations page, disclaimer

13. Ethical and Legal Considerations

- The app is an **educational prototype**. It must not be used to diagnose, treat or make medical decisions.
- No personal data is collected or stored.
- The model is trained on a limited, historical dataset and may be less accurate for groups under-represented in it.
- Emergency guidance is included: severe chest pain, breathing trouble or fainting requires immediate medical help.

14. Assumptions and Dependencies

- Users enter accurate values from real medical reports.
- The public UCI dataset remains available.
- Python 3.9+ and the listed libraries can be installed on the development machine, or Colab/Streamlit Cloud is used instead.

15. Future Roadmap

1. User accounts and stored history (SQLite/MongoDB)
2. SHAP explanations and probability calibration
3. External validation and larger combined datasets (Cleveland, Hungarian, Switzerland, Long Beach)
4. React frontend with Flask/FastAPI backend
5. Multilingual interface
6. Cloud deployment with automated retraining

16. Open Questions

1. Should the default model be chosen by cross-validated ROC-AUC (current) or by recall, to catch more disease cases?
2. Will the demo run locally or on Streamlit Community Cloud?
3. Do the dataset's category codes match the official documentation used by your guide?